

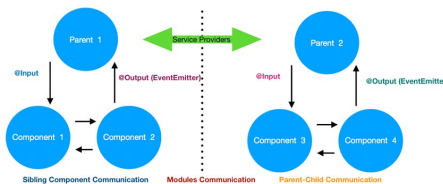
Component Communication

Angular, as a robust front-end framework, allows you to build complex and interactive web applications by creating reusable components. Components are the building blocks of an Angular application. Using components we break-down a complex application into multiple, reusable components to make the application maintenance-friendly, testable, scalable, and more-performant.

Sometimes app development requires you to send data into a component. There are few ways in which components can communicate or share data between them. And methods depend on whether the components have a Parent-child relationship between them or not. One of the key aspects of building such applications is effective communication between these components.

The communication can be between parent and child components or between sibling components. There are 3 most common ways to transfer data on our angular application.

- Parent to Child Communication (Using @Input)
- Child to Parent Communication (Using @Output)
- Sibling Communication (Using a Shared Service)
(Will be covered while angular services section.)



Parent to Child Communication (Using @Input)

If the Components have a parent-child relationship then, the parent component can pass the data to the child component using the @Input Property. This is similar to props in other frameworks. To create an Input property, use the @Input decorator.

- @Input() allows a child component to receive data from a parent component.
- It is a decorator that marks a property in the child component as bindable from outside.
- Think of it as passing arguments to a function, but here you are passing data into a component.

Syntax:

```
@Input() propertyName: type;
```

Let's do some hands-on:

- Create one basic angular application or use an existing one.
- Create two components named parentcomp and childcomp
- In parentcomp.ts create user object:

```
export class ParentcompComponent {  
  user = {  
    name: 'Ram',  
    age: 35,  
    city: 'Hyderabad'  
  };  
}
```

- In parentcomp.html create template: it will give an error as child component need to be updated.

```
<p>parentcomp works!</p>  
<app-childcomp [userData]="user"></app-childcomp>
```

Started updating as on July 18th, 2024

- Now go to childcomp.ts and change:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-childcomp',
  standalone: true,
  imports: [],
  templateUrl: './childcomp.component.html',
  styleUrls: ['./childcomp.component.css']
})
export class ChildcompComponent {
  @Input() userData!: { name: string; age: number; city: string };
  ngOnInit() {
    if (!this.userData) {
      console.error('No user data provided!');
    }
  }
}
```

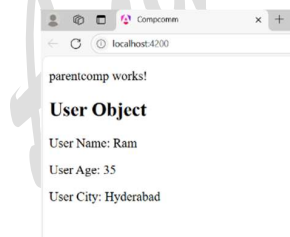
- Now go to childcomp.component.html and design template

```
<h2>User Object</h2>
<p>User Name: {{ userData.name }}</p>
<p>User Age: {{ userData.age }}</p>
<p>User City: {{ userData.city }}</p>
```

- Go to app.component.html and add

```
<app-parentcomp></app-parentcomp>
<router-outlet />
```

- And in app.component.ts check ParentcompComponent is updated in imports.
- Run and observe:



- The parent component defines a user object and passes it to the child component using property binding [userData]="user".
- The child component declares an @Input() property userData to receive the data.
- In ngOnInit(), the child component checks if userData is provided and logs an error if not, ensuring error handling.

Child to Parent Communication Using @Output()

When working with components it may be required to notify other components that something has happened. Perhaps a button has been clicked, an item has been added/removed from a list or some other important update has occurred. In this scenario components need to communicate with parent components. Angular uses the @Output decorator to enable this type of behavior.

Started updating as on July 18th, 2024

In such a situation, a child component needs to send data back to its parent and the @Output() decorator along with EventEmitter is used.

EventEmitter: is used in components with the @Output directive to emit custom events synchronously or asynchronously, and register handlers for those events by subscribing to an instance.

Let's understand this communication in a scenario where child component sending a notification to parent component.

Let's create an example demonstrating @Output

The @Output decorator in Angular is used for child-to-parent component communication. It allows a child component to emit events that a parent component can listen to and react to.

Just to keep things separate I am adding two more components named: Parent and Child in the same project.

In Child.ts

```
import { Component, EventEmitter, Output } from '@angular/core';

@Component({
  selector: 'app-child',
  imports: [],
  templateUrl: './child.html',
  styleUrls: ['./child.css']
})
export class Child {
  @Output() messageEvent = new EventEmitter<string>();
  sendMessage() {
    this.messageEvent.emit('Hello from the Child Component!');
  }
}
```

In Child.html

```
<h2>Output example Child Component!</h2>
<button (click)="sendMessage()">Send Message to Parent</button>
```

- The ChildComponent defines a property messageEvent decorated with @Output().
- messageEvent is an instance of EventEmitter<string>, meaning it will emit events with a string payload.
- The sendMessage() method is called when the button is clicked. Inside this method, this.messageEvent.emit() is used to emit the event with the string "Hello from the Child Component!".

In Parent.ts

```
import { Component } from '@angular/core';
```

```
import { Child } from "../child/child";

@Component({
  selector: 'app-parent',
  imports: [Child],
  templateUrl: './parent.html',
  styleUrls: ['./parent.css']
})
export class Parent {
  receivedMessage: string = '';
  handleMessage(message: string) {
    this.receivedMessage = message;
  }
}
```

In Parent.html

```
<h2>Output example Parent Component!</h2>
@if(receivedMessage){
  <p>Received from child: {{ receivedMessage }}</p>
}@else {
  <p>Sorry! No Message Received Yet!!</p>
<app-child (messageEvent)="handleMessage($event)"></app-child>
}
```

- The ParentComponent includes the app-child selector in its template.
- It listens for the messageEvent emitted by the child using the () event binding syntax: (messageEvent)="handleMessage(\$event)".
- When messageEvent is emitted by the ChildComponent, the handleMessage() method in the ParentComponent is called, and the emitted data (the message string) is passed as the \$event variable.
- The handleMessage() method then updates the receivedMessage property, which is displayed in the parent component's template.

In App.ts

```
import { Component, signal } from '@angular/core';
import { Parent } from "../components/parent/parent";

@Component({
  selector: 'app-root',
  imports: [Parent],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
```

```
})  
export class App {  
}
```

In App.html

```
<app-parent></app-parent>
```

Output



Exercise: Product Example

Create a file prodModel.ts, which defines a Product Model (Optional but Good Practice)

```
export interface Product {  
  name: string;  
  price: number;  
  description: string;  
}
```

Parent Component – Product List (product-list.ts)

Create Product List Component

```
>ng g c productList
```

In ProductList.ts import product interface

```
import { Product } from '../prodModel';
```

and create sample data:

```
export class ProductList {  
  // Sample product data  
  products: Product[] = [  
    { name: 'Laptop', price: 55000, description: 'High performance laptop' },  
    { name: 'Smartphone', price: 25000, description: 'Latest Android phone' },  
    { name: 'Headphones', price: 2000, description: 'Noise cancelling headphones' }  
  ];  
}
```

```
}
```

Now create cart array in same class:

```
// Cart array
cart: string[] = [];
```

Add a function onProductAdded()

```
// Function to handle product added from child
onProductAdded(productName: string) {
  this.cart.push(productName);
}
```

Add Parent Template – Product List (product-list.shtml)

```
<h2>Product List (Parent Component)</h2>

<!-- Loop through all products and pass them to child -->
@for(p of products; track $index){
  <div>
    <app-product-detail [product]="p"
    (addToCart)="onProductAdded($event)"> </app-product-detail>
  </div>
}
<hr />

<h3><img alt="shopping cart icon" data-bbox="195 578 215 595"/> Cart Items</h3>
<ul>
  @for (item of cart; track $index) {
    <li>{{ item }}</li>
  }
</ul>
```

Now create a Child Component – Product Detail (product-detail.ts)

```
>ng g c productDetails
```

Add following to .ts file class

```
import { Component, EventEmitter, Input, Output } from
'@angular/core';
import { Product } from '../prodModel';
```

```
@Component({
  selector: 'app-product-details',
  imports: [],
  templateUrl: './product-details.html',
  styleUrls: ['./product-details.css']
})
export class ProductDetails {
  @Input() product!: Product; // Receive product from parent
  @Output() addToCart = new EventEmitter<string>(); // Send event to parent

  // Function to emit product name
  addProduct() {
    this.addToCart.emit(this.product.name);
  }
}
```

Now add following to Child Template – Product Detail (product-detail.html)

```
<h2>Product List (Parent Component)</h2>

<!-- Loop through all products and pass them to child -->
@for(p of products; track $index){
<div>
  <app-product-details [product]="p"
  (addToCart)="onProductAdded($event)"></app-product-details>
</div>
}
<hr />

<h3>🛒 Cart Items</h3>
<ul>
  @for (item of cart; track $index) {
    <li>{{ item }}</li>
  }
</ul>
```

Modern Way of @Input and @Output

Angular continues to evolve with Signal-based APIs, introducing a cleaner, more declarative, and reactive approach to building components. In Angular 21, data is shared between parent and child components using the new, more type-safe `input()` and `output()` functions, which are a modern replacement for the traditional `@Input()` and `@Output()` decorators.

Let's create basic angular application and add two components named Parent and Child Components.

Using `input()` (Parent to Child communication)

The `input()` function is used to pass data from a parent component to a child component. Input values are exposed as read-only Signals in the child component.

In the Child Component (`child-component.ts`)

Import the input function:

```
import { Component, input } from '@angular/core';
```

Define the input property:

```
export class ChildComponent {  
  message = input<string>('Default Message');  
}
```

Design Template (.html) to display message from

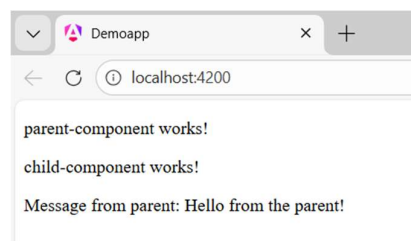
```
<p>child-component works!</p>  
<p>Message from parent: {{ message() }}</p>
```

Now in Parent Component (`parent-component.ts`)

```
export class ParentComponent {  
  parentMessage = 'Hello from the parent!';  
}
```

Bind to the input property using property binding syntax (`[[ ]]`)

```
<p>parent-component works!</p>  
<app-child-component [message]="parentMessage"></app-child-component>
```



Using output() (Child to Parent communication)

Add two component for example: Parent and Child

The output() function is used to emit custom events from the child component to the parent component, notifying the parent of changes and optionally passing data.

In child.ts Import the output function

```
import { Component, output } from '@angular/core';
```

Define the output property and use the .emit() method to raise an event

```
export class Child {  
  notifyParent = output<string>();  
  sendMessage() {  
    this.notifyParent.emit('Hello from the child!');  
  }  
}
```

In Template make one button and call sendMessage() on click event.

```
<p>child works!</p>  
<button (click)="sendMessage()">Send to Parent</button>
```

Now in Parent component listen for the output event using event binding syntax (())

In Parent component class

❖ ❖ End of Chapter ❖ ❖